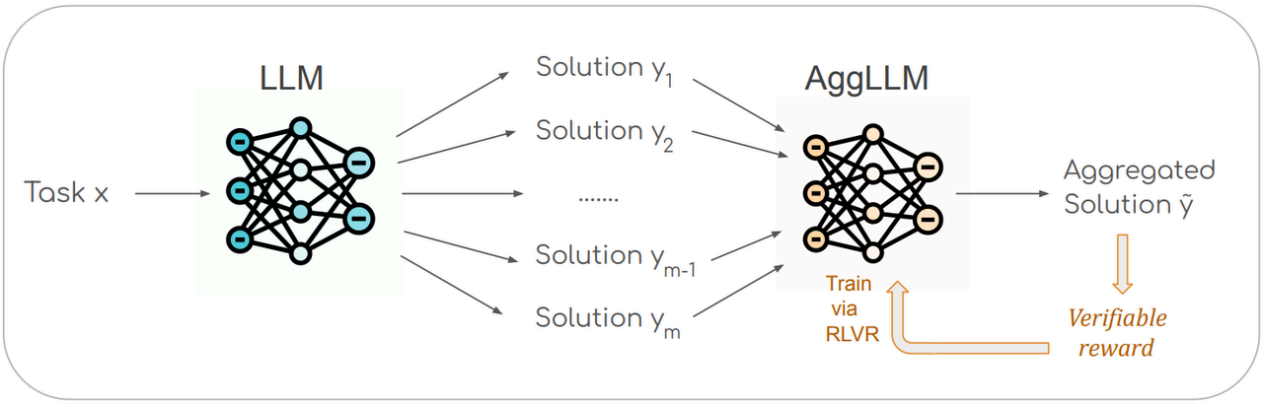
The Majority Is Not Always Right: RL Training for Solution Aggregation

- Classically, a conventional way to increase the ability of handling complicated tasks is by majority voting and reward model ranking. However, these classic methods are not very effective.
- One way to solve this problem is to use a finetuned LLM instance as an aggregatator via *reinforcement learning from verifiable rewards (RLVR)*. The responsibilities of the aggregatator are:
  - Find out and accept the correct answers.
  - Synthesis that detects mistakes. The aggregatator should find out the bugs and mistakes of the answer, use other sample answers to correct, or synthesize multiple correct answers to form a new, more completed answer.



- Methodology
  - Inference
    - Prompt used in this project:
    - The candidate solution generator model, denoted as p_θ.
    - The aggregatator p_ϕ, a non-freeze model and the policy model in the RLVR, is responsible for solution aggregation and optimization.
      - *p_ϕ can be the same model as p_θ.*
    - Let p_θ generate m candidate answers $\{y_i\}|_{i=1}^{m}$
    - Then, let p_ϕ aggregate the m answers per set.

    $\hat{y} \sim p_\phi(y|x,\ y_{1\ldots m})$
  - RLVR training data collection:
    - As RLVR is used, we need a verifiable answer. Thus, a model for ground truth answer generation is needed.
    - For each question x, we compose its ground truth answer, $D^* = \{(x, y^*)_i\}|_{i=0}^{n}$.
    - For each x, we let p_θ generate $s \cdot m$ candidate answers, where s is the # of set, and m is the # of solutions per set. We use s to increase diversity of the data by changing prompt and other augmentation methods.
    - Combine D^* with the candidate answers to form $D' = \{x, y_{1\ldots m}, y^*\}^{s \cdot n}$.
    - Difficulty management: we define a *simple task* as: most answers in $\{y_i\}|_{i=1}^{m}$ are correct;

and vice versa for a *hard task*. We construct the final training mixture by taking all hard examples and mixing in $p\%$ of easy examples, producing a balanced dataset that preserves realism while emphasizing challenging cases.
        - Combine the m answers into the final answer $\tilde{y}$.
    - Training: Optimize $p_\phi$using GRPO, where the reward is $\mathbb{I}(\hat{y} = \tilde{y})$. Like:
- Experiment Setup and Results
    - Model Implementation
        - Aggregator Model (AggLM-1.7B): initialized from Qwen3-1.7B (Instruct).
        - Training Data:
            - Source: DeepScaler dataset (approx. 40k math problems).
            - Construction: Sampled 128 independent solutions per problem (Temperature=1.5), divided into sets of m=8.
            - Data Mixture: Crucially balanced by taking all "hard" sets (majority answer wrong) and mixing in 50% "easy" sets (majority answer correct) to ensure training stability and realism.
        - Algorithm: Trained via GRPO (Group Relative Policy Optimization) for one epoch with a group size of 8 and binary verifiable rewards (r=1 if correct, else 0).
    - Evaluation Protocol
        - Datasets: Evaluated on 4 challenging math competition benchmarks: AIME24, AIME25, HMMT24, HMMT25.
            - Note: Small sample size (30 examples each) necessitates a robust sampling protocol.
        - Robust Sampling Protocol:
            - Independently sample 128 solutions per problem (Temperature=1).
            - Partition them into 16 sets (s=16) of 8 solutions (m=8) each.
            - Generate 4 aggregated answers per set.
            - Metric: Compute Pass@1 for each set, then average across all 16 sets to get the final score.
    - Baselines
        - Rule-Based: Majority Voting, Best-of-N (Oracle).
        - Model-Based Selection (Reward Models):
            - AceMath-7B & AceMath-72B: Used to score candidates; select top-1 or use Weighted Majority Voting.
        - Ablation: Prompted Aggregation (Qwen3-1.7B without RL training) to isolate the effect of the RL policy.
    - Evaluation on In-Distribution Solutions (1.7B Thinking)
        - Performance: AggLM-1.7B achieves the best performance across all four benchmarks.
        - Comparison: It consistently outperforms Majority Voting (with 3-7 point gains) and the untrained Prompted Aggregation baseline.
        - vs. Reward Models: Learned synthesis (AggLM) outperforms learned selection (Reward Models), even when RMs are much larger (72B).

    - Evaluation on Out-of-Distribution (OOD) Stronger Models (8B Thinking)
        - Generalization: AggLM-1.7B effectively generalizes to aggregating solutions from a stronger model (Qwen3-8B), despite being trained only on 1.7B data.
        - Ranking: It remains the top performer, beating both Majority Voting and AceMath reward models in this higher-quality solution space.
    - Evaluation of Non-Thinking Solutions
        - Robustness: The model generalizes to a different modality (solutions without "Chain of Thought" / thinking process).
        - Observation: In this lower-signal regime (where base solutions are weaker), Reward Models show more utility than usual, but AggLM still achieves the best overall performance by synthesizing and correcting candidates rather than just selecting them.
    - Key Takeaway
        - AggLM-1.7B demonstrates that generative aggregation (reviewing and rewriting) is superior to discriminative aggregation (voting or ranking). It is robust to different base model sizes (1.7B vs 8B) and modalities (thinking vs non-thinking), and achieves this with high token efficiency compared to simply scaling up majority voting samples.
- Work Cited
    - Paper
    - Chat History